

Mermaid Live Editor Clone - Product Specification

1. Overview

This specification defines a web-based flowchart and process-mapping tool modeled after the Mermaid Live Editor. The product enables users to write Mermaid syntax, visualize diagrams in real time, iterate quickly, and share outputs.

2. Goals

- Provide a fast, responsive editing experience for Mermaid diagrams.
- Support common diagram use cases: flowcharts, sequence diagrams, class diagrams, and process maps.
- Enable users to save, export, and share diagrams with minimal friction.
- Maintain broad browser compatibility and accessible UX.

3. Non-Goals

- Building a full collaborative multi-user editor in v1.
- Replacing general design tools (Figma, Miro) for freeform drawing.
- Supporting non-Mermaid diagram languages in v1.

4. Target Users

- Engineers documenting architecture and workflows.
- Product managers mapping business processes.
- Technical writers creating documentation diagrams.
- Students and educators learning systems/process visualization.

5. Core Features (v1)

5.1 Real-time Editor + Preview

- Split-pane UI: text editor on left, rendered output on right.
- Live rendering on input change with debouncing for performance.
- Syntax error reporting with line-level hints where possible.

5.2 Diagram Template Starter Library

- Quick-insert templates for common diagram types:
 - Flowchart
 - Sequence diagram
 - Gantt chart
 - State diagram

- Class diagram

5.3 Export + Share

- Export rendered diagram as SVG and PNG.
- Copy Mermaid source to clipboard.
- Generate shareable links using encoded diagram state.

5.4 Persistence

- Local auto-save in browser storage.
- Manual reset to blank template.

5.5 Theming

- Support default Mermaid themes (default, dark, forest, neutral).
- Preview theme change in real time.

6. Functional Requirements

1. The editor must render valid Mermaid syntax within 500 ms for medium diagrams (~300 lines) on modern desktops.
2. Invalid syntax must surface a clear error message without crashing the app.
3. Exported SVG must preserve text legibility and diagram structure.
4. Share links must fully reconstruct editor state (source + selected theme).
5. App state must persist across page refreshes by default.

7. Non-Functional Requirements

- **Performance:** First contentful load under 2.5 seconds on broadband.
- **Reliability:** Rendering failures should degrade gracefully to error panel.
- **Security:** No server-side execution of arbitrary Mermaid input for v1.
- **Accessibility:** Keyboard navigable controls and adequate color contrast.
- **Compatibility:** Latest two versions of Chrome, Firefox, Safari, Edge.

8. Architecture (High-Level)

- Frontend SPA (Svelte/React/Vue acceptable; Mermaid Live Editor uses Svelte).
- Mermaid rendering engine initialized in-browser.
- State model includes:
 - `sourceText`
 - `theme`
 - `renderStatus`
 - `shareToken`

- Optional lightweight backend for permalink storage (v2+); v1 can use URL encoding only.

9. UX Requirements

- Default layout includes:
 - Header: app name, theme picker, export menu
 - Left: code editor with line numbers and syntax highlighting
 - Right: preview canvas with zoom controls
- Error panel should be visible but non-blocking.
- Provide sample diagram on first load to reduce empty-state friction.

10. API/Integration Requirements

- No required backend APIs for v1 core functionality.
- Optional integrations:
 - GitHub Markdown copy helper
 - Embeddable iframe mode for docs portals

11. Testing Strategy

- Unit tests for:
 - state serialization/deserialization
 - share-link encoding/decoding
 - theme switching logic
- Integration tests for:
 - typing -> rendering flow
 - syntax error handling
 - export actions
- E2E smoke tests for major browsers.

12. Success Metrics

- Time-to-first-render < 1 second on median desktop session.
- Export success rate > 99%.
- Share-link open success rate > 99%.
- User retention: return usage within 7 days (tracked in analytics-enabled deployments).

13. Risks and Mitigations

- ****Risk:**** Large diagrams can cause render lag.
- **Mitigation:**** debounce input and optimize rerender boundaries.

- **Risk:** Mermaid version upgrades can introduce breaking changes.

Mitigation: pin Mermaid version and run compatibility test suite before upgrades.

- **Risk:** Browser memory spikes with very large SVG output.

Mitigation: provide render limits and user guidance.

14. Milestones (Technical Scope)

1. Project scaffold + editor/preview shell.
2. Live render loop + error handling.
3. Export/share/persistence features.
4. Theme controls + template gallery.
5. Test hardening + release prep.

15. Definition of Done

- All v1 core features implemented and tested.
- Documentation includes quick start + known limitations.
- CI pipeline passes lint, tests, and production build.
- Final QA validates required browsers and accessibility checks.